

HOMEWORK – LECTURE 3

Algorithms and Complexity Analysis

Submission. For each of Exercises 1–3, complete **two of the three** difficulty levels: E, M, and H. Complete all parts of Exercise 4 as the final integrated problem. Show the reasoning leading to every asymptotic conclusion; an answer containing only a complexity class receives limited credit. Write all answers by hand on paper. Include your full name and student ID on each page, then upload clear photos in the correct order or combine them into one PDF.

Notation. Use membership notation: for example, write $f(n) \in O(g(n))$ and $f(n) \in \Theta(g(n))$. Unless a problem states otherwise, assume that each comparison, assignment, arithmetic operation on fixed-size values, and array access takes constant time.

Exercise 1. Calculate running time from pseudocode

Common requirements for 1E, 1M, and 1H. For each chosen algorithm, let n denote the input length or the given numeric parameter. Derive a running-time function or summation and give a tight Θ bound. Show sufficient work to justify the conclusion.

Exercise 1E (Easy). Count positive values

```

FUNCTION COUNT_POSITIVE
INPUT: array values of length n
OUTPUT: the number of positive values
    count = 0
    FOR i = 0 TO n - 1 DO
        IF values[i] > 0 THEN
            count = count + 1
    RETURN count

```

Determine the best-case and worst-case running-time functions and their tight asymptotic classes.

Exercise 1M (Medium). Sieve of Eratosthenes

```

FUNCTION SIEVE_OF_ERATOSTHENES
INPUT: an integer n >= 2
OUTPUT: all prime numbers from 2 through n
    CREATE array isPrime[0..n] and set every entry to TRUE
    isPrime[0] = FALSE
    isPrime[1] = FALSE

    p = 2
    WHILE p * p <= n DO
        IF isPrime[p] == TRUE THEN
            multiple = p * p
            WHILE multiple <= n DO
                isPrime[multiple] = FALSE
                multiple = multiple + p
            p = p + 1

    RETURN every i such that isPrime[i] == TRUE

```

Derive the worst-case running-time function and give a tight Θ bound. Your analysis must account for all loop iterations and conditional tests.

Exercise 1H (Hard). Diagnose a slow simulation

A beginner team reports that the following function becomes slow when the simulation level n increases.

```
def simulation_score(n):
    score = 0
    for i in range(1, n + 1):
        for k in range(1, i * i + 1):
            if k % i == 0:
                remaining = i * i
                while remaining > 1:
                    score += 1
                    remaining //= 2
    return score
```

Derive the worst-case running-time function and give a tight Θ bound. Your analysis must account for all loop iterations and conditional tests.

Exercise 2. Simplify growth functions

Common requirements for 2E, 2M, and 2H. For each chosen function:

- find the simplest one-term function $g(n)$ such that $f(n) \in \Theta(g(n))$;
- prove the result by establishing appropriate Big- O and Big- Ω bounds.

Assume $n \geq 2$ and use any fixed logarithm base greater than 1.

Exercise 2E (Easy). Polynomial terms

Analyze

$$f(n) = 6n^2 + 4n + 20.$$

Give constants $c_1, c_2 > 0$ and a threshold n_0 that establish a two-sided Θ bound.

Exercise 2M (Medium). Logarithmic factors

Analyze

$$f(n) = 4n(\log n)^2 + 7n \log n + 3 \log(n^3) + 50.$$

Exercise 2H (Hard). Competing exponentials

Analyze

$$f(n) = n \cdot 5^n + n \cdot 3^{2n} + n^{10} \cdot 2^{n/2}.$$

Exercise 3. Compare functions and prove bounds

Common requirements for 3E and 3M. Sort the functions from slowest to fastest asymptotic growth. Use $<$ when the left function is asymptotically smaller and $=$ when two functions belong to the same Θ class. Briefly justify each non-obvious comparison.

Exercise 3E (Easy). Common growth classes

Order the following functions:

$$n^2, \quad \log n, \quad 2^n, \quad \sqrt{n}, \quad 1, \quad n \log n, \quad n.$$

Exercise 3M (Medium). Less familiar comparisons

Order the functions in each group independently.

Group	Functions
A	$n^{0.9} \log n, 100,000,000n, n^{3/2}, 1.01^n$
B	$2^{2^{20}}, n^2/\sqrt{n}, 2^{\sqrt{n}}, n^{\sqrt{n}}, 2^n$
C	$n^{\sqrt{n}}, 2^n, n^{10} \cdot 2^{n/2}, \sum_{i=1}^n (i+1)$
D	$n!, 2^n, n^{n/2}, e^{\sqrt{n}}$

Exercise 3H (Hard). Proof and counterexample

Let $f_1, f_2 : \mathbb{N} \rightarrow \mathbb{R}_{\geq 0}$ be non-negative functions.

(a) Prove that

$$f_1(n) + f_2(n) \in \Theta(\max\{f_1(n), f_2(n)\}).$$

(b) Now additionally suppose that $f_1(n) \geq f_2(n)$ for all sufficiently large n , so $f_1(n) - f_2(n)$ is eventually non-negative. Prove or disprove the statement

$$\max\{f_1(n), f_2(n)\} \in \Theta(f_1(n) - f_2(n)).$$

If it is false, give explicit functions and explain why they form a counterexample.

Exercise 4. Integrated analysis and redesign

The following algorithm multiplies two $n \times n$ matrices.

```

FUNCTION MATRIX_MULTIPLY
INPUT: two n by n matrices A and B
OUTPUT: the n by n matrix C = A times B
    CREATE matrix C with n rows and n columns

    FOR i = 0 TO n - 1 DO
        FOR j = 0 TO n - 1 DO
            total = 0
            FOR k = 0 TO n - 1 DO
                total = total + A[i][k] * B[k][j]
            C[i][j] = total
    RETURN C

```

Complete all four parts.

Part 1. Understand and validate the algorithm. State the input and output precisely. For

$$A = \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix} \quad \text{and} \quad B = \begin{bmatrix} 2 & 0 \\ 1 & 2 \end{bmatrix},$$

give the returned matrix. Explain why the algorithm is definite, finite, and correct.

Part 2. Analyze its running time. Derive $T(n)$ and give a tight Θ bound.

Part 3. Analyze its space usage. Count one unit per matrix entry and one unit per scalar variable. Total space includes both inputs and the output; auxiliary space excludes inputs and output. Derive $S_{\text{total}}(n)$ and $S_{\text{aux}}(n)$, then give tight Θ bounds.

Part 4. Redesign and compare. Suppose only the largest entry of C is required. Write language-independent pseudocode for `MAX_PRODUCT_ENTRY(A, B)` without storing the full matrix C . Analyze its time and space complexity, then compare it with `MATRIX_MULTIPLY`.